

AULA 5

Integração com Banco de Dados



OBJETIVOS DA AULA



AO FINAL DESSA AULA, VOCÊ VAI SER CAPAZ DE...

- Entender o que um ORM resolve e por que usar um
- Instalar e inicializar o Prisma num projeto NestJS
- Modelar uma entidade no schema.prisma
- Criar e aplicar sua primeira migration
- Entender quando usar SQLite e quando usar PostgreSQL



CONCEITOS DE ORM



O QUE UM ORM RESOLVE

- Manipula o banco através de objetos/classes, não SQL puro na mão
- Gera as queries, mapeia linhas do banco pra objetos tipados
- Gerencia migrations — o histórico versionado do schema do banco

TRADE-OFFS

- Vantagem: produtividade, tipagem, portabilidade entre bancos
- Cuidado: é uma camada de abstração a mais — usada sem atenção, pode esconder queries pouco eficientes
- O NestJS não força um ORM específico — se integra bem com TypeORM, Sequelize, Mongoose e Prisma
- Este curso usa Prisma, pelo foco em tipagem forte gerada automaticamente



INTRODUÇÃO AO PRISMA ORM



INSTALAÇÃO E INICIALIZAÇÃO

```
npm install prisma --save-dev
npm install @prisma/client

npx prisma init --datasource-provider sqlite
```

Esse comando cria: schema.prisma, .env e prisma.config.ts.

3 ARQUIVOS GERADOS

- schema.prisma — schema do banco (models + configuração)
- prisma.config.ts — configuração do Prisma (schema, migrations, datasource)
- .env — variáveis de ambiente, incluindo DATABASE_URL
- O Prisma atual (v7) usa "driver adapters" — um pacote específico pra cada banco (ex: better-sqlite3, pg)



MODELAGEM DE ENTIDADES



SCHEMA.PRISMA

```
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id      Int      @id @default(autoincrement())
  name    String
  email   String @unique
}
```

O MESMO USER, AGORA MODELADO

- @id marca a chave primária
- @default(autoincrement()) gera o id sozinho, como o idCounter que a gente fazia na mão
- @unique impede dois usuários com o mesmo e-mail — o banco garante isso, não só o código
- É o mesmo User que já existia como array em memória — agora com schema formal



MIGRATIONS



O QUE É UMA MIGRATION

```
npx prisma migrate dev --name init
```

Gera o SQL a partir do schema.prisma, aplica no banco de desenvolvimento, e roda "prisma generate" sozinho no final.

POR QUE ISSO IMPORTA

- Migrations são arquivos SQL versionados junto com o código — todo ambiente (dev, staging, produção) aplica o mesmo histórico
- Toda vez que o schema.prisma mudar, roda-se "migrate dev" de novo — nunca edite o banco manualmente
- npx prisma generate atualiza o Prisma Client com os tipos do schema mais recente



POSTGRESQL E SQLITE



SQLITE — DESENVOLVIMENTO

- Banco é só um arquivo local — sem servidor rodando
- Ótimo pra aprender e pra prototipar rápido
- É o que a gente está usando neste curso
- Driver adapter: `@prisma/adapters-better-sqlite3`

POSTGRESQL — PRODUÇÃO

- Precisa de um servidor de banco rodando
- Mais robusto pra concorrência e carga real
- Trocar de banco = mudar "provider" no datasource + a DATABASE_URL
- Driver adapter: `@prisma/adapters-pg`

